

**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

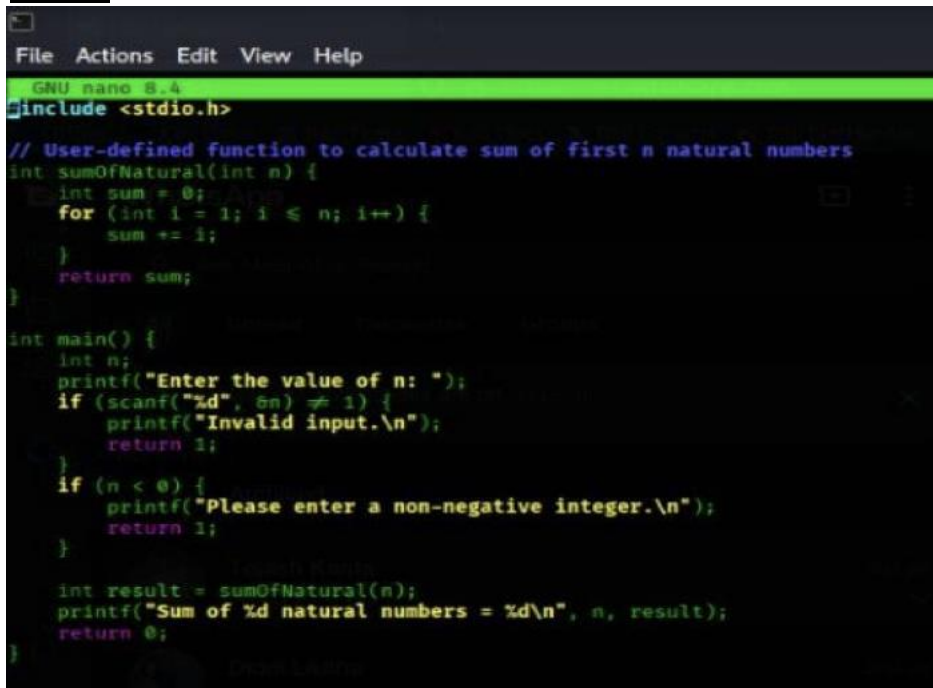
Week-1

CH.SC.U4CSE24221

TEJESH KANTA

1. Write a program to find sum of first n natural numbers using user defined function.

Code:



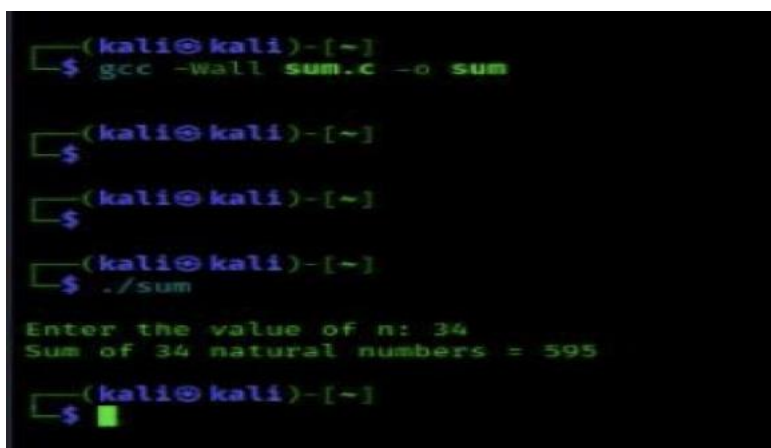
```
File Actions Edit View Help
GNU nano 2.9.4
#include <stdio.h>

// User-defined function to calculate sum of first n natural numbers
int sumOfNatural(int n) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    return sum;
}

int main() {
    int n;
    printf("Enter the value of n: ");
    if (scanf("%d", &n) != 1) {
        printf("Invalid input.\n");
        return 1;
    }
    if (n < 0) {
        printf("Please enter a non-negative integer.\n");
        return 1;
    }

    int result = sumOfNatural(n);
    printf("Sum of %d natural numbers = %d\n", n, result);
    return 0;
}
```

Output:



```
(kali@kali)-[~]
$ gcc -Wall sum.c -o sum

(kali@kali)-[~]
$

(kali@kali)-[~]
$

(kali@kali)-[~]
$ ./sum
Enter the value of n: 34
Sum of 34 natural numbers = 595

(kali@kali)-[~]
$
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$ because only a fixed number of variables are used.
- In main(), there is only **one int variable** (n).
- In sum_n(), only **two int variables** (s and i) are used.
- No arrays or dynamic memory → space stays constant.

2. Write a program to find sum of squares of first n natural numbers

Code:

```
GNU nano 8.4
#include <stdio.h>

int main() {
    int n, i;
    int sum = 0;

    printf("Enter a number: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++) {
        sum += i * i; // adding square of each number
    }

    printf("Sum of squares of first %d natural numbers = %d\n", n, sum);
    return 0;
}
```

Output:

```
(kali@kali)-[~]
$ ./sum_square

Enter a number: 2
Sum of squares of first 2 natural numbers = 5
Enter the value of n:
(kali@kali)-[~]
$
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$
- main() uses only one int (n).
- sum_sq() uses two ints (s, i).
- No arrays → constant memory.

3. Write a program to find sum of cubes of first n natural numbers

Code:

```
GNU nano 8.4
#include <stdio.h>

int main() {
    int n, i;
    long long sum = 0; // to store big values safely

    printf("Enter the value of n: ");
    scanf("%d", &n);

    // loop from 1 to n
    for(i = 1; i <= n; i++) {
        sum = sum + (long long)i * i * i; // add cube of i
    }

    printf("Sum of cubes of first %d natural numbers = %lld\n", n, sum);

    return 0;
}
```

Output:

```
(kali@kali)~[~]
$ nano sum_cubes.c

(kali@kali)~[~]
$ gcc sum_cubes.c -o sum_cubes

(kali@kali)~[~]
$ ./sum_cubes

Enter the value of n: 2
Sum of cubes of first 2 natural numbers = 9

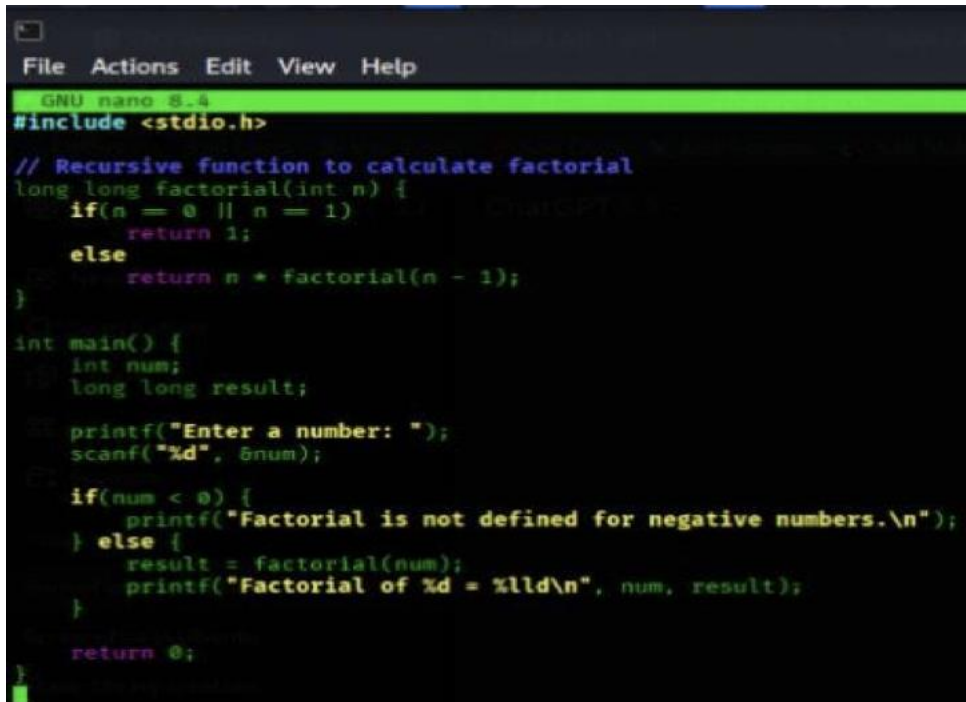
(kali@kali)~[~]
$
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$
- Only one int in main().
- Only two or three ints inside the function.
- Memory stays constant.

4. Write a program to find factorial of a given integer using recursion.

Code:



```
File Actions Edit View Help
GNU nano 2.9.4
#include <stdio.h>

// Recursive function to calculate factorial
long long factorial(int n) {
    if(n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

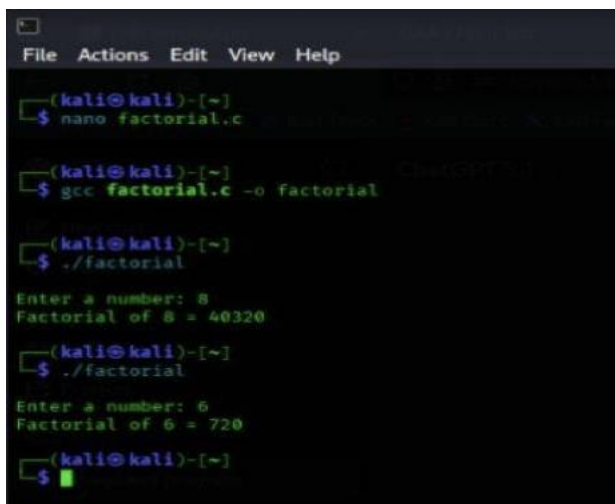
int main() {
    int num;
    long long result;

    printf("Enter a number: ");
    scanf("%d", &num);

    if(num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        result = factorial(num);
        printf("Factorial of %d = %lld\n", num, result);
    }

    return 0;
}
```

Output:



```
(kali@kali)-[~]
$ nano factorial.c

(kali@kali)-[~]
$ gcc factorial.c -o factorial

(kali@kali)-[~]
$ ./factorial

Enter a number: 8
Factorial of 8 = 40320

(kali@kali)-[~]
$ ./factorial

Enter a number: 6
Factorial of 6 = 720

(kali@kali)-[~]
$
```

Space Complexity and it's justification:

- ✓ Space Complexity: $O(n)$
- ✓ Each recursive call adds one frame to the stack.
- ✓ Maximum depth is n .
- ✓ So space grows linearly with input.

5. Write a program to transpose a 3x3 matrix.

Code:

```
GNU nano 8.4
#include <stdio.h>

int main() {
    int matrix[3][3], transpose[3][3];
    int i, j;

    // Input elements for 3x3 matrix
    printf("Enter 9 elements of the matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Finding transpose
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            transpose[j][i] = matrix[i][j];
        }
    }

    // Printing original matrix
    printf("\nOriginal Matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    // Printing transpose matrix
    printf("\nTranspose of the Matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
(kali@kali)-[~]
$ ./transpose.c
Enter 9 elements of the matrix:
1
2
3
4
5
6
7
8
9

Original Matrix:
1 2 3
4 5 6
7 8 9

Transpose of the Matrix:
1 4 7
2 5 8
3 6 9
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$ (because matrix size is fixed 3×3)

6. Write a program to find fibonacci series

Code:

```
File Actions Edit View Help
GNU nano 2.9.4
#include <stdio.h>

int main() {
    int n, i;
    int a = 0, b = 1, next;

    printf("Enter how many terms you want: ");
    scanf("%d", &n);

    printf("Fibonacci Series: ");

    for(i = 1; i <= n; i++) {
        printf("%d ", a);
        next = a + b;
        a = b;
        b = next;
    }

    printf("\n");
    return 0;
}
```

Output:

```
File Actions Edit View Help
(kali@kali)-[~]
$ nano fibonacci.c

(kali@kali)-[~]
$ gcc fibonacci.c -o fibonacci

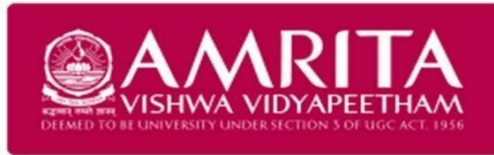
(kali@kali)-[~]
$ ./fibonacci

Enter how many terms you want: 5
Fibonacci Series: 0 1 1 2 3

(kali@kali)-[~]
$
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$
- Uses only a few ints (a, b, c, i, n).
- No arrays or recursion.
- Memory fixed $\Rightarrow$ constant space.



**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE 211

Week-2

CH.SC.U4CSE24221

TEJESH KANTA

Bubble Sort

Code:

```
#include <stdio.h>

int main() {
    int arr[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // bubble sort
    for(i = 0; i < n-1; i++) {
        for(j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    printf("Sorted array:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
amma@amma18:~$ cd Desktop
amma@amma18:~/Desktop$ gcc bubble1.c -o bubble1
amma@amma18:~/Desktop$ ./bubble1
Enter number of elements: 5
Enter the elements:
12 3 4 17 35
Sorted array:
3 4 12 17 35 amma@amma18:~/Desktop$
```

Bubble Sort

Code:

Space Complexity and it's Justification:

Time Complexity

- Best case: $O(n^2)$
→ No optimization (no early break used).
- Average case: $O(n^2)$
- Worst case: $O(n^2)$
→ Nested loops compare all adjacent pairs.

Space Complexity

- $O(1)$
→ Only one temporary variable used (in-place sorting).

Insertion Sort

Code:

```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n, i;

    printf("CH.SC.U4CSE24221-KANTA TEJESH\n");

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    insertionSort(arr, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

OUTPUT:

```
amma@amma18:~/Desktop$ gcc insertion.c -o insertion
amma@amma18:~/Desktop$ ./insertion
CH.SC.U4CSE24221-KANTA TEJESH
Enter number of elements: 7
Enter 7 elements:
15 14 16 10 18 35 23
Sorted array: 10 14 15 16 18 23 35 amma@amma18:~/Desktop$
```

Insertion Sort

Code:

Space Complexity and it's Justification:

Time Complexity

- Best: $O(n)$
Justification:
 - Already sorted array → only one comparison per element
- Average / Worst: $O(n^2)$
Justification:
 - Each element may shift through the entire sorted part

Space Complexity

- $O(1)$
Justification:
 - Uses only one temporary variable (key)

Selection Sort

Code:

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;

    for (i = 0; i < n - 1; i++) {
        minIndex = i;

        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int n, i;
    printf("CH.SC.U4CSE24221- KANTA TEJESH \n");
    printf("Enter number of elements: ");
    scanf("%d", &n);
    |
    int arr[n];

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    selectionSort(arr, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Selection Sort

Code:

```
amma@amma18:~$ cd Desktop
amma@amma18:~/Desktop$ gcc selection.c -o selection
amma@amma18:~/Desktop$ ./selection
CH.SC.U4CSE24221- KANTA TEJESH
Enter number of elements: 5
Enter 5 elements:
12 65 43 32 54
Sorted array: 12 32 43 54 65 amma@amma18:~/Desktop$
```

Space Complexity and it's Justification:

Time Complexity

- Best / Average / Worst: $O(n^2)$

Justification:

- Always compares every element to find minimum
- Number of comparisons does not depend on input order

Space Complexity

- $O(1)$

Justification:

- Sorting done **in-place**, no extra memory used

Bucket Sort

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n, i, j;

    printf("CH.SC.U4CSE24221-KANTA TEJESH\n");

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```

Bucket Sort

Code:

```
int max = arr[0];
for (i = 1; i < n; i++) {
    if (arr[i] > max)
        max = arr[i];
}

int buckets[MAX][n];
int bucketCount[MAX];

for (i = 0; i < MAX; i++)
    bucketCount[i] = 0;

for (i = 0; i < n; i++) {
    int bucketIndex = (arr[i] * MAX) / (max + 1);
    buckets[bucketIndex][bucketCount[bucketIndex]++] = arr[i];
}

for (i = 0; i < MAX; i++) {
    if (bucketCount[i] > 0)
        insertionSort(buckets[i], bucketCount[i]);
}

int index = 0;
for (i = 0; i < MAX; i++) {
    for (j = 0; j < bucketCount[i]; j++) {
        arr[index++] = buckets[i][j];
    }
}

// Print sorted array
printf("Sorted array: ");
for (i = 0; i < n; i++)
```

```
    printf("%d ", arr[i]);
```

```
    return 0;
```

```
}
```

OUTPUT:

```
amma@amma18:~/Desktop$ ./bucket
CH.SC.U4CSE24221-KANTA TEJESH
Enter number of elements: 5
Enter 5 elements:
9 8 10 6 3
Sorted array: 3 6 8 9 10 amma@amma18:~/Desktop$
```

Space Complexity and it's Justification:

Time Complexity

- Best case: $O(n)$
→ Elements uniformly distributed, minimal shifting inside buckets.

Bucket Sort

Code:

- Average case: $O(n)$
→ Each bucket has few elements; insertion is fast.
- Worst case: $O(n^2)$
→ All elements fall into one bucket → insertion sort behavior.

Space Complexity

- $O(n)$
→ Buckets array `buckets[10][n]` + count array.

Heap Sort

Code:

```
#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    int i, temp;

    for (i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (i = n - 1; i >= 0; i--) {
        temp = arr[0];
```

Heap Sort

Code:

```
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

int main() {
    int n, i;

    printf("CH.SC.U4CSE24221-KANTA TEJESH\n");

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

OUTPUT:

```
CH.SC.U4CSE24221-KANTA TEJESH
Enter number of elements: 5
Enter 5 elements: 45 56 24 54 63
Sorted array: 24 45 54 56 63
```

Space Complexity and it's Justification:

Time Complexity

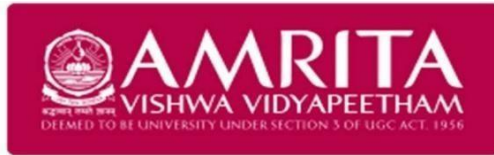
Heap Sort

Code:

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$
→ Heap construction $O(n)$ + n heapify operations $O(\log n)$.

Space Complexity

- $O(\log n)$
→ Due to recursive heapify() call stack.



**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

Week-3

CH.SC.U4CSE24221

TEJESH KANTA

BFS

Code:

```
C bfs.c > [?] n
1  #include <stdio.h>
2
3  #define MAX 10
4
5  int queue[MAX], front = -1, rear = -1;
6  int visited[MAX] = {0};
7  int adj[MAX][MAX];
8  int n;|
9
10 void enqueue(int v) {
11     if (rear == MAX - 1)
12         return;
13     if (front == -1)
14         front = 0;
15     queue[++rear] = v;
16 }
17
18 int dequeue() {
19     if (front == -1 || front > rear)
20         return -1;
21     return queue[front++];
22 }
23
24 void bfs(int start) {
25     int i, v;
26     enqueue(start);
27     visited[start] = 1;
28
29     while (front <= rear) {
30         v = dequeue();
31         printf("%d ", v);
32
33         for (i = 0; i < n; i++) {
34             if (adj[v][i] == 1 && visited[i] == 0) {
35                 enqueue(i);
36                 visited[i] = 1;
37             }
38         }
39     }
40 }
```

BFS

Code:

```
37     }
38 }
39 }
40 }
41
42 int main() {
43     int i, j, start;
44
45     printf("CH.U4CSE24221-KANTA TEJESH\n");
46
47     printf("Enter number of vertices: ");
48     scanf("%d", &n);
49
50     printf("Enter adjacency matrix:\n");
51     for (i = 0; i < n; i++)
52         for (j = 0; j < n; j++)
53             scanf("%d", &adj[i][j]);
54
55     printf("Enter starting vertex: ");
56     scanf("%d", &start);
57
58     printf("BFS Traversal: ");
59     bfs(start);
60
61     return 0;
62 }
63
```

OUTPUT:

```
[kantatejesh@KANTAs-MacBook-Air ~ % cd Desktop
[kantatejesh@KANTAs-MacBook-Air Desktop % cd Website
[kantatejesh@KANTAs-MacBook-Air Website % gcc bfs.c -o bfs
[kantatejesh@KANTAs-MacBook-Air Website % ./bfs
CH.U4CSE24221-KANTA TEJESH
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
Enter starting vertex: 1
BFS Traversal: 1 0 3 2 4 %
```

BFS

Code:

Space Complexity and it's Justification:

Time Complexity

- $O(V^2)$

Justification:

- Uses adjacency matrix
- For each vertex, all V vertices are checked

Space Complexity

- $O(V)$

Justification:

- Queue array + visited array store at most V vertices

DFS

Code:

```
C dfs.c > ...
1  #include <stdio.h>
2
3  #define MAX 10
4
5  int visited[MAX] = {0};
6  int adj[MAX][MAX];
7  int n;
8
9  void dfs(int v) {
10     int i;
11     printf("%d ", v);
12     visited[v] = 1;
13
14     for (i = 0; i < n; i++) {
15         if (adj[v][i] == 1 && visited[i] == 0) {
16             dfs(i);
17         }
18     }
19 }
20
21 int main() {
22     int i, j, start;
23
24     printf("CH.U4CSE24221-KANTA TEJESH\n");
25
26     printf("Enter number of vertices: ");
27     scanf("%d", &n);
28
29     printf("Enter adjacency matrix:\n");
30     for (i = 0; i < n; i++)
31         for (j = 0; j < n; j++)
32             scanf("%d", &adj[i][j]);
33
34     printf("Enter starting vertex: ");
35     scanf("%d", &start);
36
37     printf("DFS Traversal: ");
38     dfs(start);
39
40     return 0;
41 }
```

OUTPUT:

DFS

Code:

```
kantatejesh@KANTAs-MacBook-Air Website % ./dfs
CH.U4CSE24221-KANTA TEJESH
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
Enter starting vertex: 1
DFS Traversal: 1 0 2 4 3 %
```

Space Complexity and it's Justification:

Time Complexity

- $O(V^2)$

Justification:

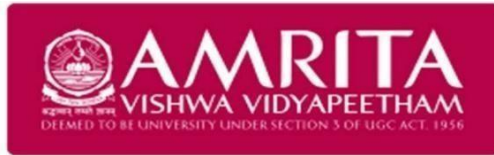
- Adjacency matrix representation
- Each DFS call scans all vertices

Space Complexity

- $O(V)$

Justification:

- Recursive call stack + visited array



**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

Week-4

CH.SC.U4CSE24221

TEJESH KANTA

1.PRIMS

CODE:

```
C prims.c > main()
1  #include <stdio.h>
2  #define INF 999
3
4  int main() {
5      int n, cost[10][10];
6      int visited[10] = {0};
7      int i, j, ne = 1;
8      int min, a = 0, b = 0;
9      int mincost = 0;
10
11     printf("TEJESH KANTA-CH.SC.U4CSE24221");
12
13     printf("Enter number of vertices: ");
14     scanf("%d", &n);
15
16     printf("Enter cost adjacency matrix:\n");
17     for (i = 1; i <= n; i++) {
18         for (j = 1; j <= n; j++) {
19             scanf("%d", &cost[i][j]);
20             if (cost[i][j] == 0)
21                 cost[i][j] = INF;
22         }
23     }
24
25     visited[1] = 1;
26
27     printf("\nEdges in Minimum Spanning Tree:\n");
28
29     while (ne < n) {
30         min = INF;
31         for (i = 1; i <= n; i++) {
32             if (visited[i]) {
33                 for (j = 1; j <= n; j++) {
34                     if (!visited[j] && cost[i][j] < min) {
35                         min = cost[i][j];
36                         a = i;
37                         b = j;
```

```

37         b = j;
38     }
39 }
40 }
41 }
42
43     printf("%d -> %d = %d\n", a, b, min);
44     mincost += min;
45     visited[b] = 1;
46     ne++;
47 }
48
49 printf("\nMinimum cost = %d\n", mincost);
50 return 0;
51 }

```

OUTPUT:

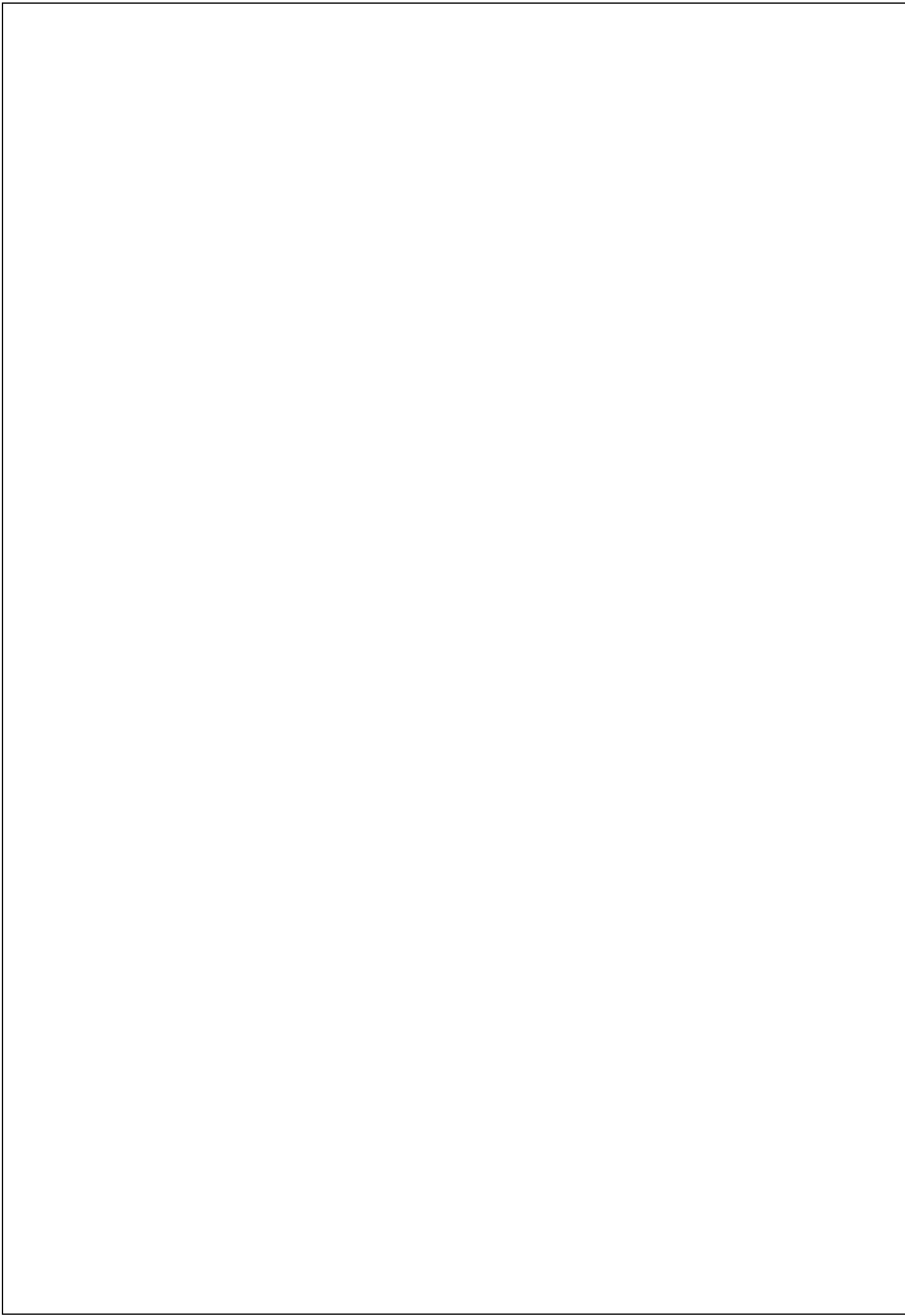
```

[kantatejesh@KANTAs-MacBook-Air Desktop % cd Website
[kantatejesh@KANTAs-MacBook-Air Website % gcc prims.c -o prims
[kantatejesh@KANTAs-MacBook-Air Website % ./prims
TEJESH KANTA-CH.SC.U4CSE24221Enter number of vertices: 4
Enter cost adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0

Edges in Minimum Spanning Tree:
1 -> 4 = 5
4 -> 3 = 4
1 -> 2 = 10

Minimum cost = 19

```



2.KRUSKAL

CODE:

```
C kruskal.c > main()
1  #include <stdio.h>
2
3  int parent[10];
4
5  int find(int i) {
6      while (parent[i])
7          i = parent[i];
8      return i;
9  }
10
11 int uni(int i, int j) {
12     if (i != j) {
13         parent[j] = i;
14         return 1;
15     }
16     return 0;
17 }
18
19 int main() {
20     int n, cost[10][10];
21     int i, j, ne = 1;
22     int min, a = 0, b = 0;
23     int mincost = 0;
24     printf("TEJESH KANTA-CH.SC.U4CSE24221");
25
26     printf("Enter number of vertices: ");
27     scanf("%d", &n);
28
29     printf("Enter cost adjacency matrix:\n");
30     for (i = 1; i <= n; i++) {
31         for (j = 1; j <= n; j++) {
32             scanf("%d", &cost[i][j]);
33             if (cost[i][j] == 0)
34                 cost[i][j] = 999;
35         }
36     }
37 }
```

```

37
38     printf("\nEdges in Minimum Spanning Tree
39
40     while (ne < n) {
41         min = 999;
42         for (i = 1; i <= n; i++) {
43             for (j = 1; j <= n; j++) {
44                 if (cost[i][j] < min) {
45                     min = cost[i][j];
46                     a = i;
47                     b = j;
48                 }
49             }
50         }
51
52         int u = find(a);
53         int v = find(b);
54
55         if (uni(u, v)) {
56             printf("%d -> %d = %d\n", a, b,
57                 mincost += min;
58                 ne++;
59         }
60
61         cost[i][j] = cost[j][i] = 999;

```

OUTPUT:

```

[kantatejesh@KANTAs-MacBook-Air Website % ./kruska
TEJESH KANTA-CH.SC.U4CSE24221Enter number of vert
Enter cost adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in Minimum Spanning Tree:
1 -> 2 = 2
2 -> 3 = 3
2 -> 5 = 5
1 -> 4 = 6

```


3.AVL

CODE:

```
Users > kantatejesh > Desktop > DAA LAB > TASK4 > C AVL.c > ...
1  //CH.SC.U4CSE24221
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  struct Node {
6      int key;
7      struct Node *left, *right;
8      int height;
9  };
10
11  int max(int a, int b) {
12      return (a > b) ? a : b;
13  }
14
15  int getHeight(struct Node *n) {
16      return (n == NULL) ? 0 : n->height;
17  }
18
19  struct Node* newNode(int key) {
20      struct Node* node = (struct Node*)malloc(sizeof(struct Node));
21      node->key = key;
22      node->left = node->right = NULL;
23      node->height = 1;
24      return node;
25  }
26
27  struct Node* rightRotate(struct Node *y) {
28      struct Node *x = y->left;
29      struct Node *T2 = x->right;
30
31      x->right = y;
32      y->left = T2;
33
34      y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
35      x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
36
37      return x;
38  }
39
40  struct Node* leftRotate(struct Node *x) {
```

```

41     struct Node *y = x->right;
42     struct Node *T2 = y->left;
43
44     y->left = x;
45     x->right = T2;
46
47     x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
48     y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
49
50     return y;
51 }
52
53 struct Node* balanceTree(struct Node* node) {
54     if (node == NULL)
55         return NULL;
56
57     node->left = balanceTree(node->left);
58     node->right = balanceTree(node->right);
59
60     node->height = 1 + max(getHeight(node->left), getHeight(node->right));
61
62     int balance = getHeight(node->left) - getHeight(node->right);
63
64     if (balance > 1 && getHeight(node->left->left) >= getHeight(node->left->right))
65         return rightRotate(node);
66
67     if (balance > 1 && getHeight(node->left->left) < getHeight(node->left->right)) {
68         node->left = leftRotate(node->left);
69         return rightRotate(node);
70     }
71
72     if (balance < -1 && getHeight(node->right->right) >= getHeight(node->right->left))
73         return leftRotate(node);
74
75     if (balance < -1 && getHeight(node->right->right) < getHeight(node->right->left)) {
76         node->right = rightRotate(node->right);
77         return leftRotate(node);
78     }
79 }

```

```

80     return node;
81 }
82
83 void printGivenLevel(struct Node* root, int level) {
84     if (root == NULL)
85         return;
86
87     if (level == 1)
88         printf("%d ", root->key);
89     else {
90         printGivenLevel(root->left, level - 1);
91         printGivenLevel(root->right, level - 1);
92     }
93 }
94
95 void printLevelOrder(struct Node* root) {
96     int h = getHeight(root);
97     for (int i = 1; i <= h; i++)
98         printGivenLevel(root, i);
99 }
100
101 int main() {
102
103     struct Node *root = newNode(157);
104     root->left = newNode(110);
105     root->left->right = newNode(147);
106     root->left->right->left = newNode(122);
107     root->left->right->right = newNode(149);
108     root->left->right->right->right = newNode(151);
109     root->left->right->left->left = newNode(111);
110     root->left->right->left->right = newNode(141);
111     root->left->right->left->left->right = newNode(112);
112     root->left->right->left->right->left = newNode(123);
113     root->left->right->left->right->left->right = newNode(133);
114     root->left->right->left->right->left->left = newNode(117);
115
116     root = balanceTree(root);
117
118     printf("AVL Level Order: ");

```

OUTPUT:

AVL Level Order: 123 110 157 122 147 111 117 141 149 112
133 151

CH.SC.U4CSE24221

Space Complexity and it's Justification:

Time Complexity

$O(n)$

Justification: Each node is visited once; height check and rotations take $O(1)$ time.

Space Complexity

$O(h) \rightarrow O(\log n)$ (balanced), $O(n)$ (skewed)

Justification: Space used only by recursion stack, proportional to tree height.

4. RED

BLACK

TREE

CODE:

```
Users > kantatejesh > Desktop > DAA LAB > TASK4 > C RB.c > ...
1  //CH.SC.U4CSE24221
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  enum Color { RED, BLACK };
6
7  struct Node {
8      int data;
9      enum Color color;
10     struct Node *left, *right, *parent;
11 };
12
13 struct Node* newNode(int data) {
14     struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
15     temp->data = data;
16     temp->left = temp->right = temp->parent = NULL;
17     temp->color = RED;
18     return temp;
19 }
20
21 void rotateLeft(struct Node** root, struct Node* x) {
22     struct Node* y = x->right;
23     x->right = y->left;
24     if (y->left != NULL)
25         y->left->parent = x;
26
27     y->parent = x->parent;
28
29     if (x->parent == NULL)
30         *root = y;
31     else if (x == x->parent->left)
32         x->parent->left = y;
33     else
34         x->parent->right = y;
35
36     y->left = x;
37     x->parent = y;
38 }
39
40 void rotateRight(struct Node** root, struct Node* y) {
```

```

40 void rotateRight(struct Node** root, struct Node* y) {
41     struct Node* x = y->left;
42     y->left = x->right;
43     if (x->right != NULL)
44         x->right->parent = y;
45
46     x->parent = y->parent;
47
48     if (y->parent == NULL)
49         *root = x;
50     else if (y == y->parent->left)
51         y->parent->left = x;
52     else
53         y->parent->right = x;
54
55     x->right = y;
56     y->parent = x;
57 }
58
59 void fixViolation(struct Node** root, struct Node* z) {
60     while (z != *root && z->parent->color == RED) {
61
62         if (z->parent == z->parent->parent->left) {
63             struct Node* y = z->parent->parent->right;
64
65             if (y != NULL && y->color == RED) {
66                 z->parent->color = BLACK;
67                 y->color = BLACK;
68                 z->parent->parent->color = RED;
69                 z = z->parent->parent;
70             } else {
71                 if (z == z->parent->right) {
72                     z = z->parent;
73                     rotateLeft(root, z);
74                 }
75                 z->parent->color = BLACK;
76                 z->parent->parent->color = RED;
77                 rotateRight(root, z->parent->parent);
78             }
79         } else {

```

```

79         } else {
80             struct Node* y = z->parent->parent->left;
81
82             if (y != NULL && y->color == RED) {
83                 z->parent->color = BLACK;
84                 y->color = BLACK;
85                 z->parent->parent->color = RED;
86                 z = z->parent->parent;
87             } else {
88                 if (z == z->parent->left) {
89                     z = z->parent;
90                     rotateRight(root, z);
91                 }
92                 z->parent->color = BLACK;
93                 z->parent->parent->color = RED;
94                 rotateLeft(root, z->parent->parent);
95             }
96         }
97     }
98     (*root)->color = BLACK;
99 }
100
101 int height(struct Node* node) {
102     if (node == NULL)
103         return 0;
104     int l = height(node->left);
105     int r = height(node->right);
106     return (l > r ? l : r) + 1;
107 }
108
109 void printGivenLevel(struct Node* root, int level) {
110     if (root == NULL)
111         return;
112
113     if (level == 1)
114         printf("%d ", root->data);
115     else {
116         printGivenLevel(root->left, level - 1);

```

```
128
129  int main() {
130
131      struct Node* root = newNode(157);
132      root->color = BLACK;
133
134      struct Node* n110 = newNode(110);
135      root->left = n110; n110->parent = root;
136
137      struct Node* n147 = newNode(147);
138      n110->right = n147; n147->parent = n110;
139
140      struct Node* n122 = newNode(122);
141      n147->left = n122; n122->parent = n147;
142
143      struct Node* n149 = newNode(149);
144      n147->right = n149; n149->parent = n147;
145
146      struct Node* n111 = newNode(111);
147      n122->left = n111; n111->parent = n122;
148
149      struct Node* n141 = newNode(141);
150      n122->right = n141; n141->parent = n122;
151
152      struct Node* n151 = newNode(151);
153      n149->right = n151; n151->parent = n149;
154
155      struct Node* n112 = newNode(112);
156      n111->right = n112; n112->parent = n111;
157
158      struct Node* n123 = newNode(123);
159      n141->left = n123; n123->parent = n141;
160
161      struct Node* n117 = newNode(117);
162      n112->left = n117; n117->parent = n112;
163
164      struct Node* n133 = newNode(133);
165      n123->right = n133; n133->parent = n123;
166
```

```

166
167     fixViolation(&root, n147);
168     fixViolation(&root, n111);
169     fixViolation(&root, n141);
170     fixViolation(&root, n151);
171     fixViolation(&root, n117);
172     fixViolation(&root, n133);
173
174     printf("Red-Black Level Order: ");
175     printLevelOrder(root);
176     printf("\n");
177     printf("CH.SC.U4CSE24221\n");
178     return 0;
179 }
180
181

```

OUTPUT:

```

Red-Black Level Order: 122 | 111 147 | 110 112 133 151 |
    117 123 141 149 157 |
CH.SC.U4CSE24221

```

Space Complexity and Its Justification:

Time Complexity

$O(\log n)$

Justification:

Red-Black Tree height is always $O(\log n)$.

Insertion fix (fixViolation) does at most constant rotations and recolorings while moving up the tree.

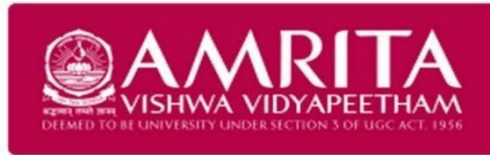
Space Complexity

$O(h) \rightarrow O(\log n)$

Justification:

Uses recursion only for traversal/height, and recursion depth equals tree height.

No extra data structures are used.



**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

Week-5

CH.SC.U4CSE24221

TEJESH KANTA

Merge Sort

Code:

```
C mergesort.c > ...
1  #include <stdio.h>
2
3
4  void merge(int arr[], int left, int mid, int right)
5  {
6      int i, j, k;
7      int n1 = mid - left + 1;
8      int n2 = right - mid;
9
10     int L[n1], R[n2];
11
12     for (i = 0; i < n1; i++)
13         L[i] = arr[left + i];
14     for (j = 0; j < n2; j++)
15         R[j] = arr[mid + 1 + j];
16
17     i = 0;
18     j = 0;
19     k = left;
20
21
22     while (i < n1 && j < n2)
23     {
24         if (L[i] <= R[j])
25             arr[k++] = L[i++];
26         else
27             arr[k++] = R[j++];
28     }
29
30
31     while (i < n1)
32         arr[k++] = L[i++];
33
34     while (j < n2)
35         arr[k++] = R[j++];
36 }
```

```

37
38
39 void mergeSort(int arr[], int left, int right)
40 {
41     if (left < right)
42     {
43         int mid = (left + right) / 2;
44
45         mergeSort(arr, left, mid);
46         mergeSort(arr, mid + 1, right);
47         merge(arr, left, mid, right);
48     }
49 }
50
51
52 int main()
53 {
54     int arr[100], n, i;
55
56     printf("Enter number of elements: ");
57     scanf("%d", &n);
58
59     printf("Enter array elements:\n");
60     for (i = 0; i < n; i++)
61         scanf("%d", &arr[i]);
62
63     mergeSort(arr, 0, n - 1);
64
65     printf("Sorted array:\n");
66     for (i = 0; i < n; i++)
67         printf("%d ", arr[i]);
68
69     return 0;
70 }
71

```

Output:

```

[kantatejesh@KANTAs-MacBook-Air Website % gcc mergesort.c -o mergesort
[kantatejesh@KANTAs-MacBook-Air Website % ./mergesort
Enter number of elements: 10
Enter array elements:
23 34 10 31 22 29 47 56 67 56
Sorted array:
10 22 23 29 31 34 47 56 56 67 %
kantatejesh@KANTAs-MacBook-Air Website %

```

Space Complexity and its justification:

- Time Complexity (All cases): $O(n \log n)$
- Reason: Array is divided into halves recursively $\rightarrow \log n$ levels
- Merge Cost: At each level, n elements are merged $\rightarrow O(n)$ work
- Space Complexity: $O(n)$
- Reason: Temporary array $b[]$ is used during merging (recursion stack $O(\log n)$ is smaller)

Quick Sort

Code:

```
1  #include <stdio.h>
2
3  void swap(int *a, int *b)
4  {
5      int temp = *a;
6      *a = *b;
7      *b = temp;
8  }
9
10
11 int partition(int arr[], int low, int high)
12 {
13     int pivot = arr[high];
14     int i = low - 1;
15
16     for (int j = low; j < high; j++)
17     {
18         if (arr[j] < pivot)
19         {
20             i++;
21             swap(&arr[i], &arr[j]);
22         }
23     }
24     swap(&arr[i + 1], &arr[high]);
25     return i + 1;
26 }
27
28
29 void quickSort(int arr[], int low, int high)
30 {
31     if (low < high)
32     {
33         int pi = partition(arr, low, high);
34
35         quickSort(arr, low, pi - 1);
36         quickSort(arr, pi + 1, high);
37     }
```

```
37     }
38 }
39
40
41 int main()
42 {
43     int arr[100], n;
44
45     printf("Enter number of elements: ");
46     scanf("%d", &n);
47
48     printf("Enter array elements:\n");
49     for (int i = 0; i < n; i++)
50         scanf("%d", &arr[i]);
51
52     quickSort(arr, 0, n - 1);
53
54     printf("Sorted array:\n");
55     for (int i = 0; i < n; i++)
56         printf("%d ", arr[i]);
57
58     return 0;
59 }
60
61
```

Output:

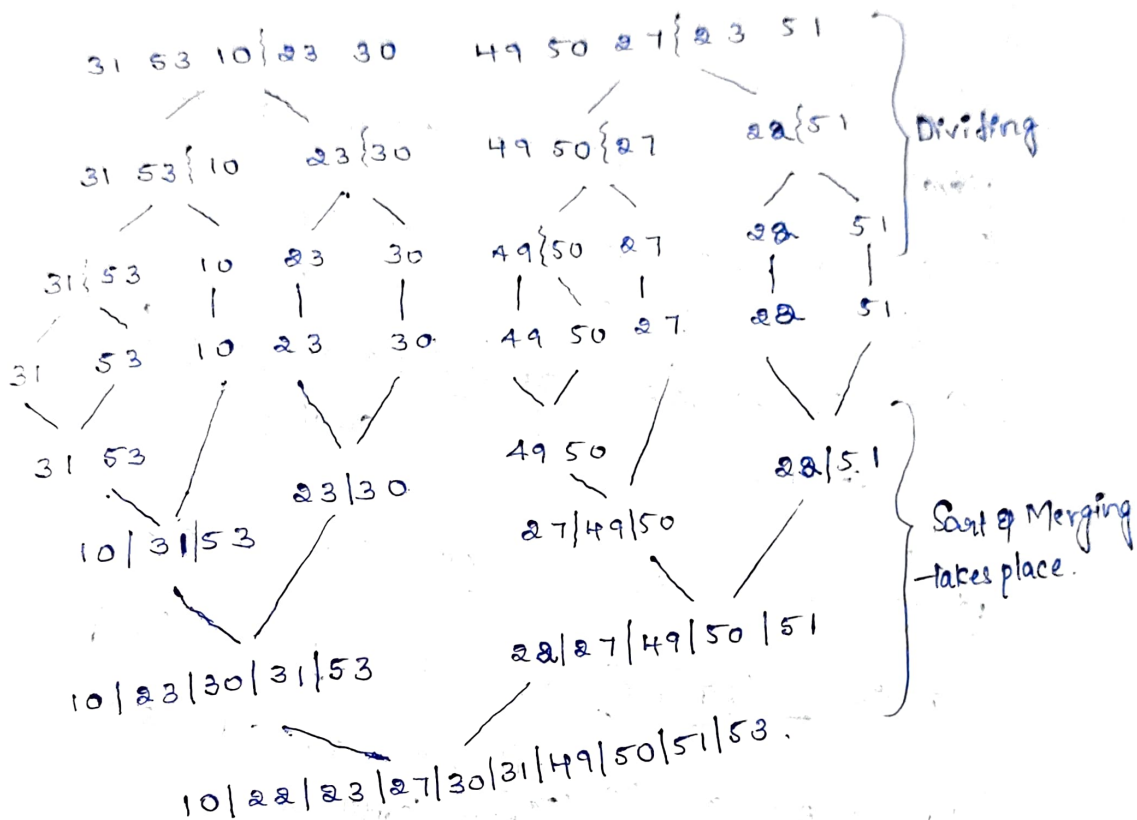
```
[kantatejesh@KANTAs-MacBook-Air ~ % cd Desktop
[kantatejesh@KANTAs-MacBook-Air Desktop % cd Website
[kantatejesh@KANTAs-MacBook-Air Website % gcc quicksort.c -o quicksort
[kantatejesh@KANTAs-MacBook-Air Website % ./quicksort
Enter number of elements: 7
Enter array elements:
12 34 56 23 31 10 8
Sorted array:
8 10 12 23 31 34 56 %
```

Space Complexity and its justification:

- ✓ Best & Average Time Complexity: $O(n \log n)$
- ✓ Worst Time Complexity: $O(n^2)$ (when pivot is smallest/largest → unbalanced partitions)
- ✓ Reason: Partitioning takes $O(n)$ and recursion depth is $\log n$ (best/avg) or n (worst)
- ✓ Space Complexity (Average): $O(\log n)$ due to recursion stack
- ✓ Space Complexity (Worst): $O(n)$ when recursion becomes linear

(1) 31 53 10 | 23 30 | 49 50 27 | 22 51
 left mid mid+1 right

Divide by using mid using Divide.



merge Sort (arr, low, mid, high) {

l → mid

mid+1 → right

temp

arr = [];

left = low;

right = mid+1;

while (left <= mid && right <= ^{high}mid+1) {

if (arr[left] <= mid && arr[right] <= ^{high}mid+1) {

(or) if (arr[left] <= arr[right]) {

temp.arr[left]

left++;

else

temp.arr[right].

right++;

}

is on left ✓

while (left <= mid) {

temp.arr[left]

left;

}

is on right ✓

while (right <= high) {

temp.arr[right]

right;

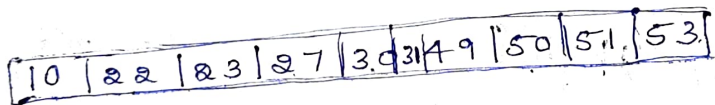
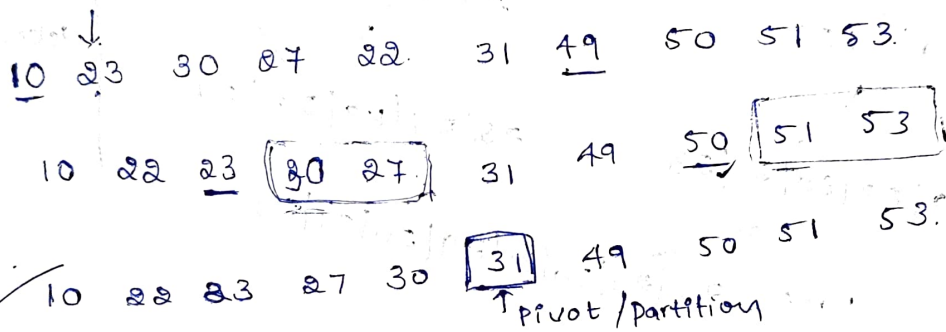
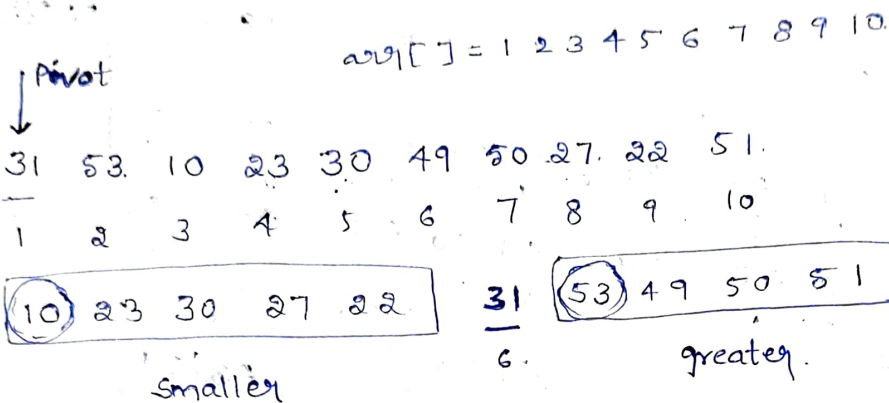
}

for (i = high $\rightarrow$ low) {

arr[i] = temp[i - low];

}

(a)



qs(low, partition - 1)

qs(high, partition + 1)

qs(arr, low, high) {

if (low < high) {

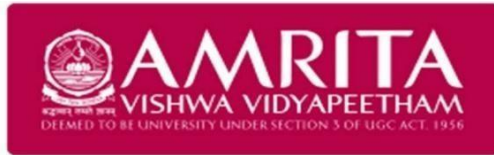
partition = qs(arr, low, high);

qs(arr, low, partition - 1);

qs(arr, partition + 1, high);

}

}



**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

Week-6

CH.SC.U4CSE24221

TEJESH KANTA

1. Job Scheduling

Code:

```
#include <stdio.h>

#include <stdlib.h>

typedef struct {
    int id;
    int deadline;
    int profit;
} Job;

int compare(const void *a, const void *b) {
    Job *j1 = (Job *)a;
    Job *j2 = (Job *)b;
    return j2->profit - j1->profit;
}

int main() {
    int n;
    printf("Enter number of jobs: ");
    scanf("%d", &n);

    Job jobs[n];

    for(int i = 0; i < n; i++) {
        printf("Enter deadline and profit for job %d: ", i+1);
        scanf("%d %d", &jobs[i].deadline, &jobs[i].profit);
        jobs[i].id = i + 1;
    }

    qsort(jobs, n, sizeof(Job), compare);

    int maxDeadline = 0;
    for(int i = 0; i < n; i++)
        if(jobs[i].deadline > maxDeadline)
            maxDeadline = jobs[i].deadline;

    int slot[maxDeadline];
    for(int i = 0; i < maxDeadline; i++)
        slot[i] = -1;

    int totalProfit = 0;

    for(int i = 0; i < n; i++) {
        for(int j = jobs[i].deadline - 1; j >= 0; j--) {
            if(slot[j] == -1) {
                slot[j] = jobs[i].id;
                totalProfit += jobs[i].profit;
                break;
            }
        }
    }

    printf("\nScheduled Jobs: ");
    for(int i = 0; i < maxDeadline; i++)
        if(slot[i] != -1)
            printf("%d ", slot[i]);

    printf("\nTotal Profit = %d\n", totalProfit);
```

```
return 0;  
}
```

OUTPUT:

```
Enter number of jobs: 4  
Enter deadline and profit for job 1: 10  
20  
Enter deadline and profit for job 2: 20 20  
Enter deadline and profit for job 3: 30 30  
Enter deadline and profit for job 4: 40 10  
  
Scheduled Jobs: J1 J2 J3 J4  
Total Profit = 80
```

Time Complexity: $O(n^2)$

Justification: The algorithm initially applies Bubble Sort with nested loops to arrange the jobs by profit, which requires $O(n^2)$ time. After sorting, it again uses nested loops where each of the n jobs checks the slot array (up to the maximum deadline d) to locate a free slot. If the maximum deadline d is of the order of n , the overall time complexity stays $O(n^2)$.

Space Complexity: $O(n + d)$

Justification: Two dynamic arrays, result and slot, are created with size equal to the maximum deadline (d). It also maintains an array of n Job structures. Since the implementation is iterative and does not use recursion, there is no extra stack memory usage. Hence, the auxiliary space depends on both the number of jobs and the deadline range.

2. Huffman Coding

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
    char data;
    unsigned freq;
    struct Node *left, *right;
};

struct Node* newNode(char data, unsigned freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}

struct MinHeap {
    unsigned size;
    struct Node* arr[MAX];
};

void swap(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}

void heapify(struct MinHeap* heap, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < heap->size && heap->arr[left]->freq < heap->arr[smallest]->freq)
        smallest = left;

    if (right < heap->size && heap->arr[right]->freq < heap->arr[smallest]->freq)
        smallest = right;

    if (smallest != i) {
        swap(&heap->arr[i], &heap->arr[smallest]);
        heapify(heap, smallest);
    }
}

struct Node* extractMin(struct MinHeap* heap) {
    struct Node* temp = heap->arr[0];
    heap->arr[0] = heap->arr[--heap->size];
    heapify(heap, 0);
    return temp;
}

void insertHeap(struct MinHeap* heap, struct Node* node) {
    int i = heap->size++;
    while (i && node->freq < heap->arr[(i-1)/2]->freq) {
        heap->arr[i] = heap->arr[(i-1)/2];
        i = (i-1)/2;
    }
}
```

```

heap->arr[i] = node;
}

struct Node* buildTree(char data[], int freq[], int size) {
    struct MinHeap heap;
    heap.size = 0;

    for (int i = 0; i < size; i++)
        heap.arr[heap.size++] = newNode(data[i], freq[i]);

    for (int i = (heap.size-1)/2; i >= 0; i--)
        heapify(&heap, i);

    while (heap.size > 1) {
        struct Node* left = extractMin(&heap);
        struct Node* right = extractMin(&heap);

        struct Node* top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;

        insertHeap(&heap, top);
    }

    return extractMin(&heap);
}

void printCodes(struct Node* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }

    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }

    if (!root->left && !root->right) {
        printf("%c: ", root->data);
        for (int i = 0; i < top; i++)
            printf("%d", arr[i]);
        printf("\n");
    }
}

int main() {
    char data[] = {'A','B','C','D','E','F'};
    int freq[] = {5,9,12,13,16,45};
    int size = sizeof(data)/sizeof(data[0]);

    struct Node* root = buildTree(data, freq, size);

    int arr[MAX], top = 0;
    printf("Huffman Codes:\n");
    printCodes(root, arr, top);

    return 0;
}

```

OUTPUT:

- kantatejesh@KANTAs-MacBook-Air ~ % gcc -o huffman huffman.c
- kantatejesh@KANTAs-MacBook-Air ~ % ./huffman
Huffman Codes:
F: 0
C: 100
D: 101
A: 1100
B: 1101
E: 111

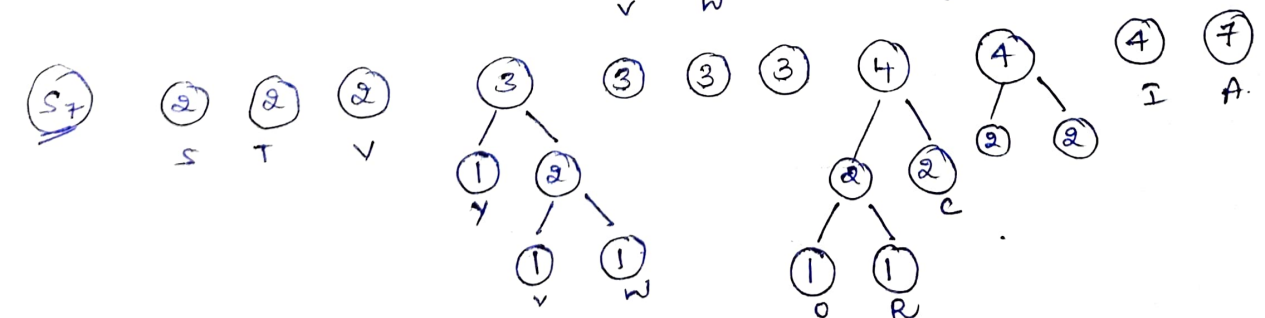
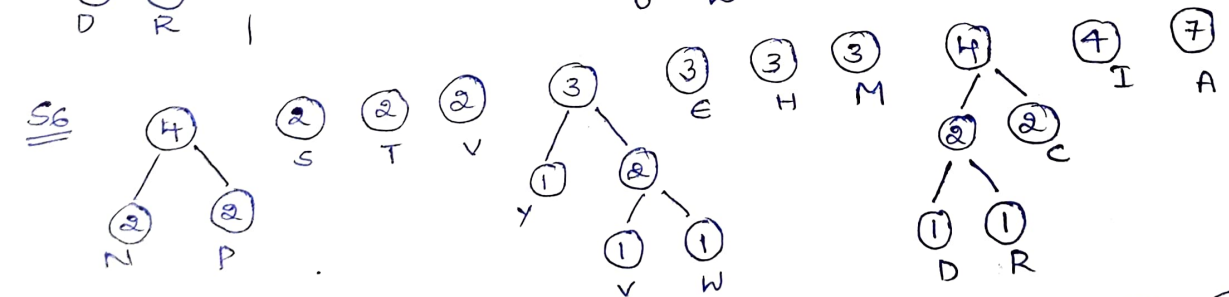
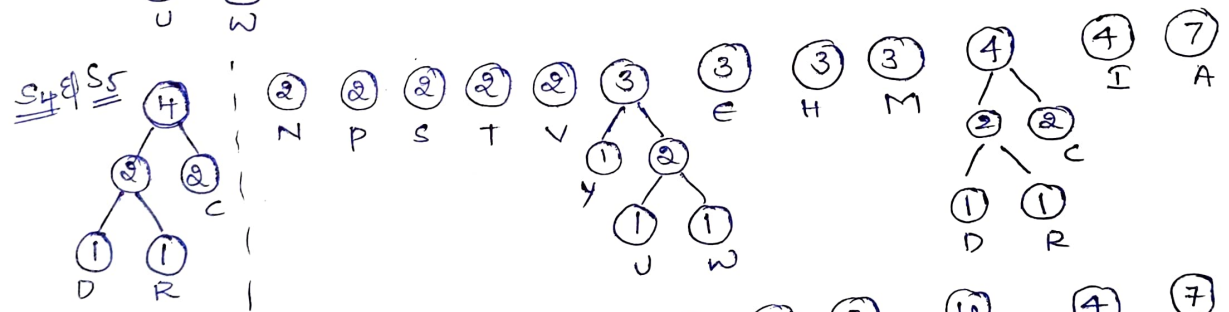
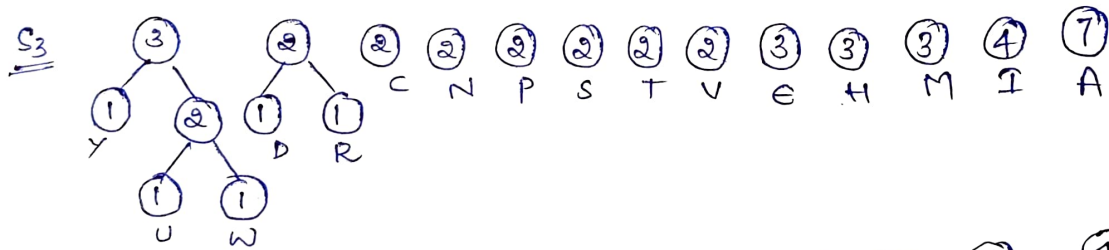
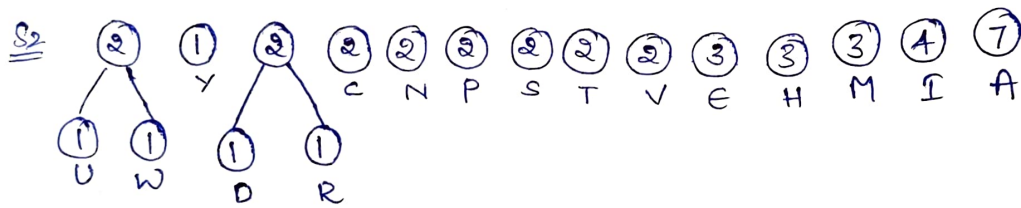
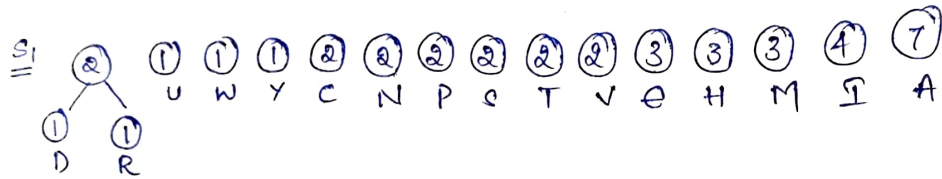
Q Using Huffman Coding arrive max encoding bits for phase

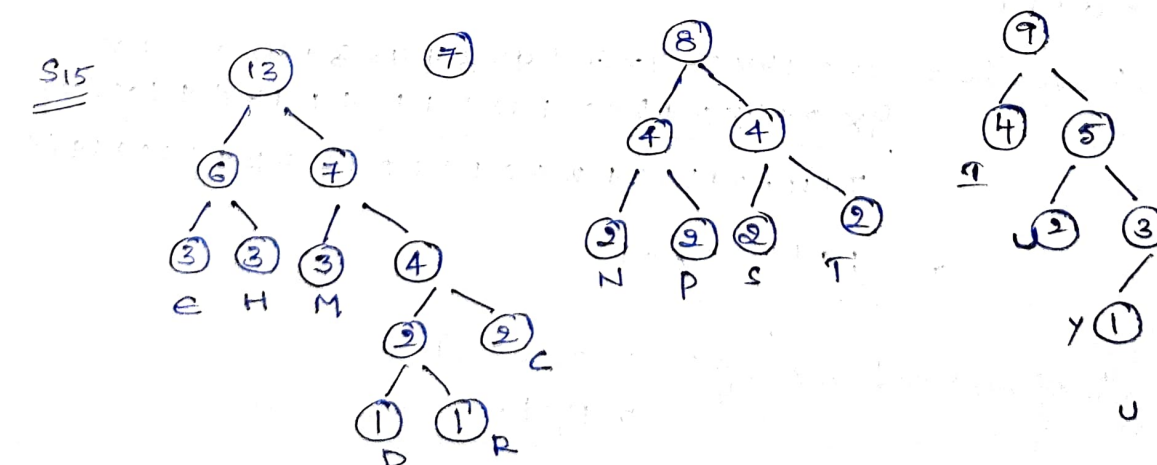
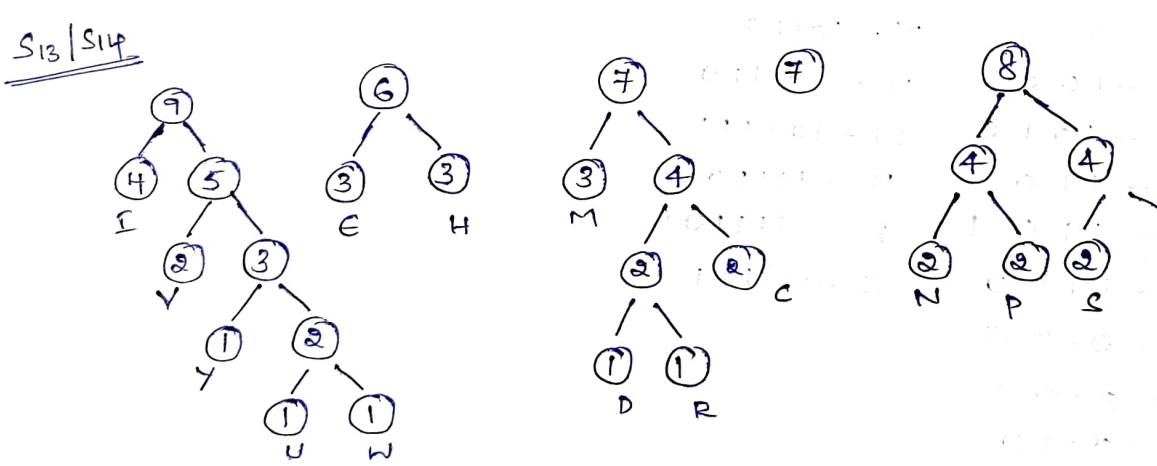
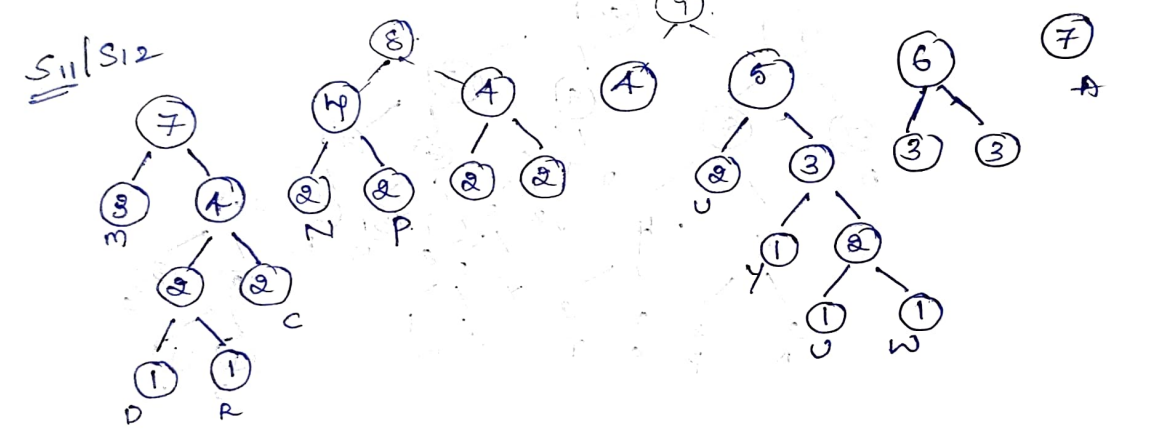
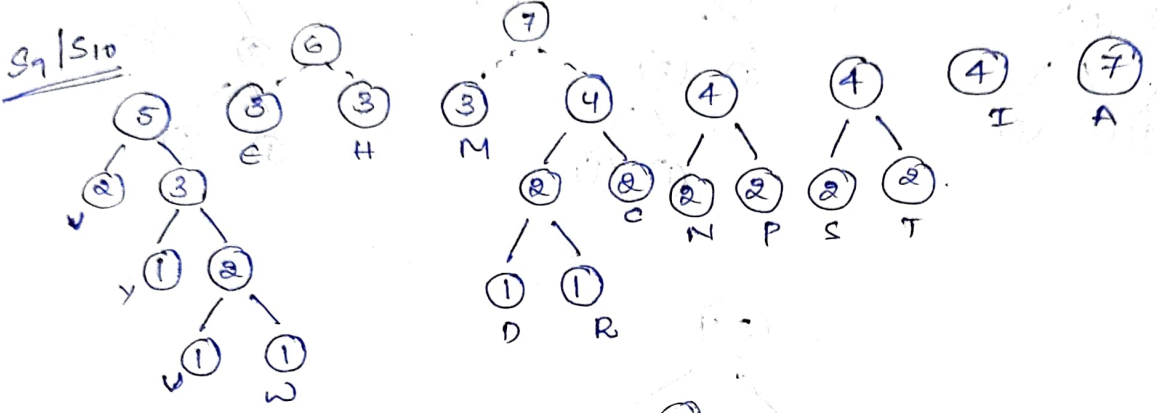
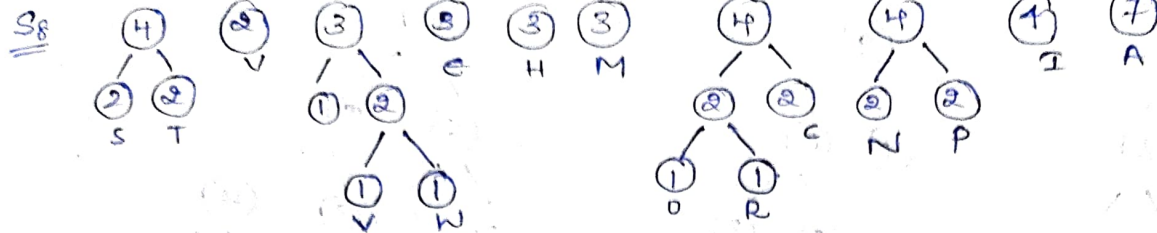
AMRITA VISHWA VIDYAPEETHAM CHENNAI CAMPUS.

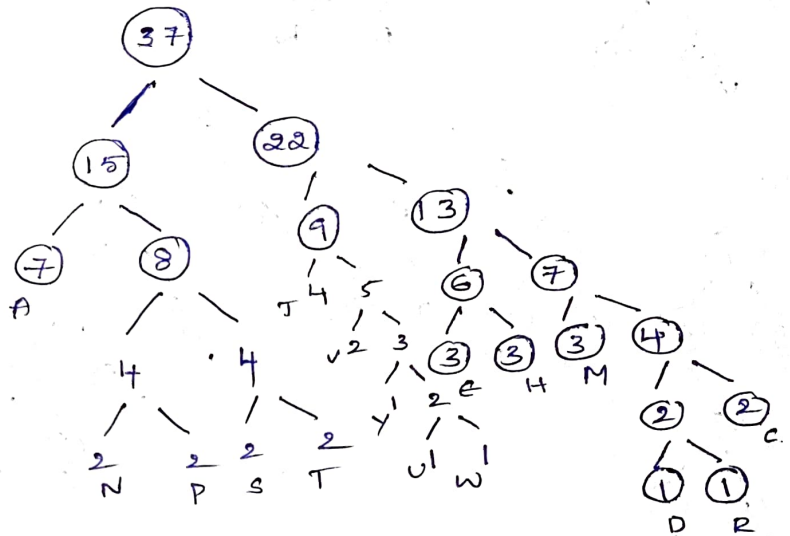
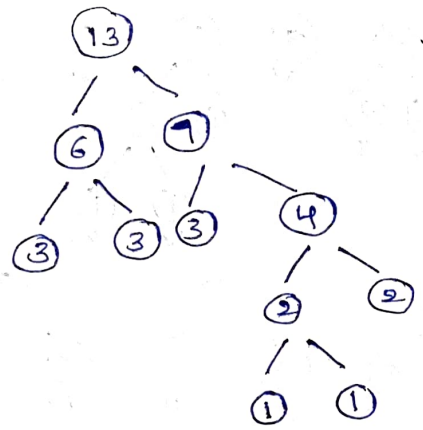
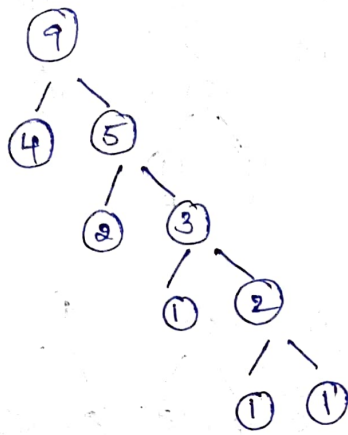
A M R I T A V I S H W D Y P E C N U
7 3 1 4 2 2 2 3 1 1 1 2 3 2 2 1

Arrange in ascending order.

① ① ① ① ① ② ② ② ② ② ② ③ ③ ③ ④ ⑦
D R U W Y C N P S T V E H M I A






$$\gamma = 10110.$$

U - 101110

W-101111

D - 111100

R - 111101

C - 11111

$$\begin{aligned} \text{Avg code length} &= \frac{7 \times 2 + 4 \times 3 + 4 \times 3 + 4 \times 3 + 4 \times 3 + 4 \times 2 + 4 \times 2 + 4 \times 2 + 4 \times 2 + 5 \times 2 + 1 \times 5 + 1 \times 6 + 1 \times 6 + 1 \times 6 + 1 \times 6}{37} \\ &= \frac{141}{37} = 3.81 \end{aligned}$$

length of encoded message $\Rightarrow 37 \times 3.81$
 $\Rightarrow 141$

2) Job scheduling.

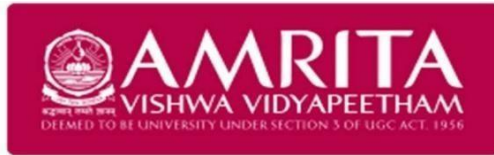
Let there be 14 jobs for sequencing whose profits are
 $36, 21, 28, 19, 21, 13, 28, 25, 18, 20, 27, 22, 14, 26$.

Times are $5, 8, 3, 4, 4, 9, 12, 14, 2, 7, 5, 1, 6, 3$.

D.O $P \{36, 28, 28, 27, 26, 25, 22, 21, 21, 20, 19, 18, 14, 13\}$.

$T \{5, 3, 12, 5, 3, 14, 1, 8, 4, 7, 4, 2, 6, 9\}$

J_{12}	J_9	J_3	J_5	J_1	J_{13}	J_{10}	J_2	J_6			J_7		J_8	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

Week-7

CH.SC.U4CSE24221

TEJESH KANTA

1 . Quick sort by different pivots and quick select

CODE:

```
C quicksort.c > main()
1 // CH.SC.U4CSE24221
2 // Implement Quick Sort using three different pivot choices: first element, middle element, and last element
3
4 #include <stdio.h>
5
6 void swap(int *a,int *b)
7 {
8     int t=*a;
9     *a=*b;
10    *b=t;
11 }
12
13 int partitionLast(int arr[],int low,int high)
14 {
15     int pivot=arr[high];
16     int i=low-1;
17
18     for(int j=low;j<high;j++)
19     {
20         if(arr[j]<pivot)
21         {
22             i++;
23             swap(&arr[i],&arr[j]);
24         }
25     }
26
27     swap(&arr[i+1],&arr[high]);
28     return i+1;
29 }
30
31 int partitionFirst(int arr[],int low,int high)
32 {
33     int pivot=arr[low];
34     int i=low+1;
35     int j=high;
36
37     while(1)
38     {
39         while(i<=high && arr[i]<=pivot) i++;
40         while(arr[j]>pivot) j--;
41
42         if(i>=j) break;
43
44         swap(&arr[i],&arr[j]);
45     }
}
```

```
46
47     swap(&arr[low],&arr[j]);
48     return j;
49 }
50
51 int partitionMiddle(int arr[],int low,int high)
52 {
53     int mid=(low+high)/2;
54     swap(&arr[mid],&arr[high]);
55     return partitionLast(arr,low,high);
56 }
57
58 void quickSortLast(int arr[],int low,int high)
59 {
60     if(low<high)
61     {
62         int pi=partitionLast(arr,low,high);
63         quickSortLast(arr,low,pi-1);
64         quickSortLast(arr,pi+1,high);
65     }
66 }
67
68 void quickSortFirst(int arr[],int low,int high)
69 {
70     if(low<high)
71     {
72         int pi=partitionFirst(arr,low,high);
73         quickSortFirst(arr,low,pi-1);
74         quickSortFirst(arr,pi+1,high);
75     }
76 }
77
78 void quickSortMiddle(int arr[],int low,int high)
79 {
80     if(low<high)
81     {
82         int pi=partitionMiddle(arr,low,high);
83         quickSortMiddle(arr,low,pi-1);
84         quickSortMiddle(arr,pi+1,high);
85     }
86 }
87
88 void printArray(int arr[],int n)
89 {
```

```

87
88  void printArray(int arr[],int n)
89  {
90      for(int i=0;i<n;i++)
91          printf("%d ",arr[i]);
92      printf("\n");
93  }
94
95  int main()
96  {
97      printf("CH.SC.U4CSE24221\n");
98
99      int n;
100     scanf("%d",&n);
101
102     int arr1[n],arr2[n],arr3[n];
103
104     for(int i=0;i<n;i++)
105     {
106         scanf("%d",&arr1[i]);
107         arr2[i]=arr1[i];
108         arr3[i]=arr1[i];
109     }
110
111     quickSortFirst(arr1,0,n-1);
112     quickSortMiddle(arr2,0,n-1);
113     quickSortLast(arr3,0,n-1);
114
115     printArray(arr1,n);
116     printArray(arr2,n);
117     printArray(arr3,n);
118
119     return 0;
120 }

```

OUTPUT

```

kantatejesh@KANTAs-MacBook-Air TASK7 % gcc quicksort.c -o quicksort
kantatejesh@KANTAs-MacBook-Air TASK7 % ./quicksort
CH.SC.U4CSE24221
5
8 3 5 6 4
3 4 5 6 8
3 4 5 6 8
3 4 5 6 8
kantatejesh@KANTAs-MacBook-Air TASK7 % 

```

2. QUICK SELECT

CODE:

```
C quickselect.c > swap(int *, int *)
1 // CH.SC.U4CSE24221
2 //Quick Select algorithm
3
4 #include <stdio.h>
5
6 void swap(int *a,int *b)
7 {
8     int t=*a;
9     *a=*b;
10    *b=t;
11 }
12
13 int partition(int arr[],int low,int high)
14 {
15     int pivot=arr[high];
16     int i=low;
17
18     for(int j=low;j<high;j++)
19     {
20         if(arr[j]<=pivot)
21         {
22             swap(&arr[i],&arr[j]);
23             i++;
24         }
25     }
26
27     swap(&arr[i],&arr[high]);
28     return i;
29 }
30
31 int quickSelect(int arr[],int low,int high,int k)
32 {
33     if(low<=high)
34     {
35         int pi=partition(arr,low,high);
36
37         if(pi==k)
38             return arr[pi];
39         else if(pi>k)
40             return quickSelect(arr,low,pi-1,k);
41         else
42             return quickSelect(arr,pi+1,high,k);
43     }
44     return -1;
45 }
```

```

46
47     int main()
48     {
49         printf("CH.SC.U4CSE24221\n");
50
51         int n;
52         scanf("%d",&n);
53
54         int arr[n];
55
56         for(int i=0;i<n;i++)
57             scanf("%d",&arr[i]);
58
59         int k;
60         scanf("%d",&k);
61
62         int result=quickSelect(arr,0,n-1,k-1);
63
64         printf("%d",result);
65
66         return 0;
67     }

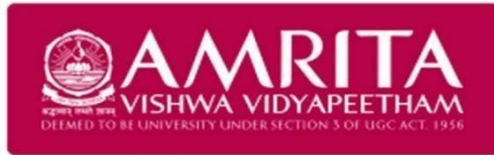
```

OUTPUT

```

kantatejesh@KANTAs-MacBook-Air TASK7 % gcc quickselect.c -o quickselect
kantatejesh@KANTAs-MacBook-Air TASK7 % ./quickselect
CH.SC.U4CSE24221
6
7 10 4 5 20 15
5
15
kantatejesh@KANTAs-MacBook-Air TASK7 % 

```

**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

Week-8

CH.SC.U4CSE24221

TEJESH KANTA

1. Huffman Coding

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
char data;
unsigned freq;
struct Node *left, *right;
};

struct Node* newNode(char data, unsigned freq) {
struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
temp->left = temp->right = NULL;
temp->data = data;
temp->freq = freq;
return temp;
}

struct MinHeap {
unsigned size;
struct Node* arr[MAX];
};

void swap(struct Node** a, struct Node** b) {
struct Node* t = *a;
*a = *b;
*b = t;
}

void heapify(struct MinHeap* heap, int i) {
int smallest = i;
int left = 2*i + 1;
int right = 2*i + 2;

if (left < heap->size && heap->arr[left]->freq < heap->arr[smallest]->freq)
smallest = left;

if (right < heap->size && heap->arr[right]->freq < heap->arr[smallest]->freq)
smallest = right;

if (smallest != i) {
swap(&heap->arr[i], &heap->arr[smallest]);
heapify(heap, smallest);
}
}

struct Node* extractMin(struct MinHeap* heap) {
struct Node* temp = heap->arr[0];
heap->arr[0] = heap->arr[--heap->size];
heapify(heap, 0);
return temp;
}

void insertHeap(struct MinHeap* heap, struct Node* node) {
int i = heap->size++;
while (i && node->freq < heap->arr[(i-1)/2]->freq) {
heap->arr[i] = heap->arr[(i-1)/2];
i = (i-1)/2;
}
```

```

heap->arr[i] = node;
}

struct Node* buildTree(char data[], int freq[], int size) {
    struct MinHeap heap;
    heap.size = 0;

    for (int i = 0; i < size; i++)
        heap.arr[heap.size++] = newNode(data[i], freq[i]);

    for (int i = (heap.size-1)/2; i >= 0; i--)
        heapify(&heap, i);

    while (heap.size > 1) {
        struct Node* left = extractMin(&heap);
        struct Node* right = extractMin(&heap);

        struct Node* top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;

        insertHeap(&heap, top);
    }

    return extractMin(&heap);
}

void printCodes(struct Node* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }

    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }

    if (!root->left && !root->right) {
        printf("%c: ", root->data);
        for (int i = 0; i < top; i++)
            printf("%d", arr[i]);
        printf("\n");
    }
}

int main() {
    char data[] = {'A','B','C','D','E','F'};
    int freq[] = {5,9,12,13,16,45};
    int size = sizeof(data)/sizeof(data[0]);

    struct Node* root = buildTree(data, freq, size);

    int arr[MAX], top = 0;
    printf("Huffman Codes:\n");
    printCodes(root, arr, top);

    return 0;
}

```

OUTPUT:

```
● kantatejesh@KANTAs-MacBook-Air ~ % gcc -o huffman huffman.c
● kantatejesh@KANTAs-MacBook-Air ~ % ./huffman
Huffman Codes:
F: 0
C: 100
D: 101
A: 1100
B: 1101
E: 111
```